

시스템구현 아키텍처

담당자	서은 박
날짜	2026년 3월 29일
상태	완료



프로젝트 핵심 정리

A. Trip-level layer

한 번의 여행 안에서

- 예산 관리
- 소비 기록
- OCR 자동 입력
- 공유 여행 협업
- 경로 기록
- 여행 종료 후 리포트 생성

B. User-level layer

사용자의 여러 여행 데이터를 누적해서

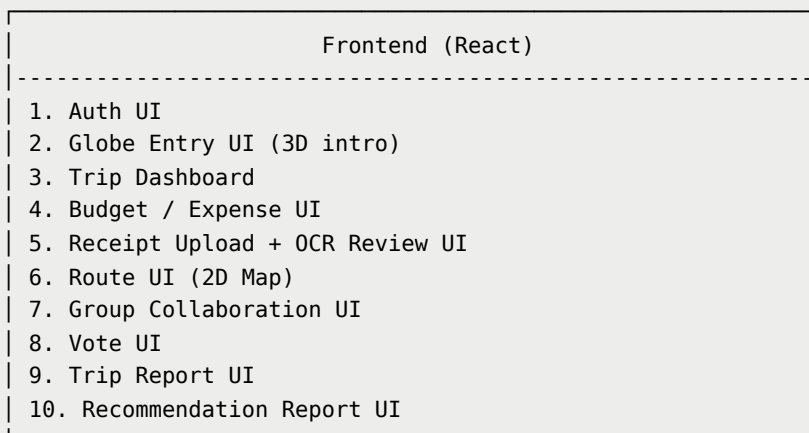
- 여행 스타일 분석
- 선호 소비 패턴 분석
- 다음 여행지 추천 리포트 생성

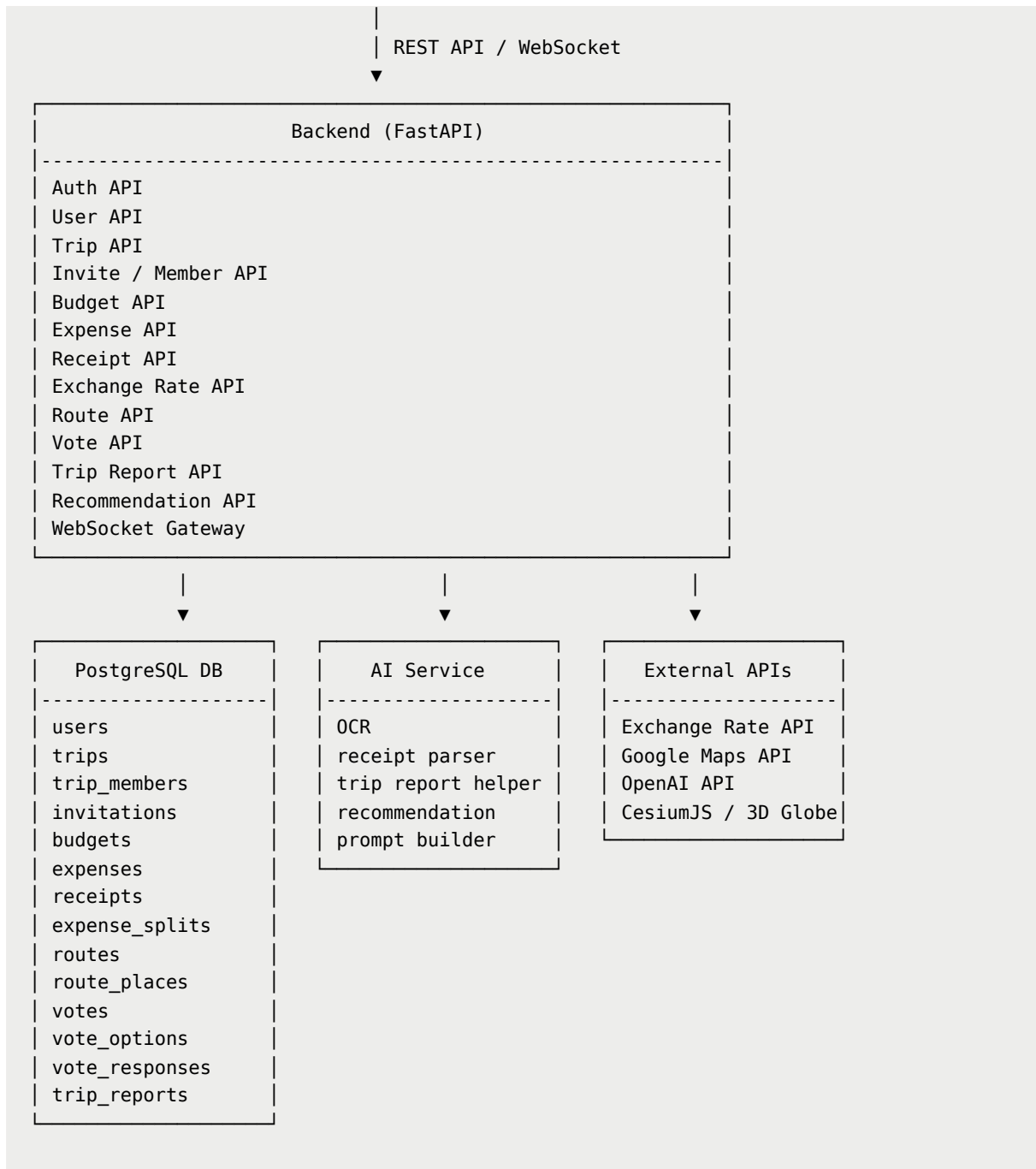


설계 원칙

1. 여행은 owner 1명이 생성
 - a. 다른 사용자는 초대받아 editor 권한으로 참여
2. 그룹 여행에서 expense는 personal / shared로 구분
 - a. personal expense는 작성자만 볼 수 있음
 - b. shared expense는 그룹 전체가 볼 수 있음
3. 공동 소비 정산은 자동이 아니라 사용자 요청 시에만 split 계산
4. 3D는 경로 편집이 아니라 첫 진입 화면의 globe 경험
5. 실제 여행지 기록과 경로 관리는 2D map 기반
6. OCR은 별도 AI service로 분리 가능한 구조: 모델 크기가 커질 것을 생각해 따로 구분
 - 추후 모델 크기가 작다면 fastApi 하나로 합치는 것도 생각해놓기
7. LLM 기반 AI는 두 갈래
 - 여행 종료 후 Trip Report
 - 사용자 누적 이력 기반 Recommendation Report

1. 전체 시스템 구조





2. 전체 흐름 시각화

2.1. 사용자 전체 흐름

[회원가입/로그인]
↓
[3D Globe Entry]
↓

[내 여행 선택 or 새 여행 생성]

↓

[Trip Dashboard]

- └ 예산 설정
- └ 소비 입력 / OCR 업로드
- └ 멤버 초대 / 그룹 협업
- └ 2D 지도 기반 경로 기록
- └ 투표 / 공유 활동
- └ 여행 종료 후 Trip Report 생성
- └ 누적 데이터 기반 Recommendation Report 생성

2.2 백엔드 기준 데이터 흐름

사용자 입력

↓

React Frontend

↓

FastAPI

- └ JWT 인증 확인
- └ 권한 확인
- └ DB CRUD
- └ AI Service 호출
- └ 외부 API 호출
- └ WebSocket 이벤트 브로드캐스트

↓

Frontend 상태 갱신

3. 기능별 상세 아키텍처



총 8개의 모듈 구조

Module 1. Authentication & User: 로그인, 회원가입, 토큰 관리, 사용자 정보

Module 2. Trip & Membership: 여행 생성, 여행 소유자, 초대코드, 멤버 권한

Module 3. Budget & Expense: 예산, 개인/공동 지출, 환율 적용, 과소비 분석 (예산 위험도)

Module 4. OCR Pipeline: 영수증 업로드, OCR, JSON 파싱, 사용자 검수

Module 5. Route & Places: 2D 지도, 장소 기록, 경로 순서 관리

Module 6. Collaboration: 멤버 참여, 실시간 shared expense, 실시간 Vote

Module 7. Trip Report: 한 여행이 끝난 뒤 여행 요약 리포트 생성

Module 8. Recommendation Engine: 여러 여행 누적해서 사용자 스타일 분석 및 다음 여행지 추천

▼ 3.1. Auth/ User 아키텍처

1. 목적

- 사용자 식별
- 여행 접근 통제
- personal expense 접근 통제
- recommendation report의 주체 식별

2. 흐름

```

[회원가입]
Frontend
  ↓
POST /auth/signup
  ↓
FastAPI
  ├── 이메일 중복 검사
  ├── password_hash 생성
  └── users 저장
  ↓
응답

[로그인]
Frontend
  ↓
POST /auth/login
  ↓
FastAPI
  ├── user 조회
  ├── 비밀번호 검증
  └── JWT 발급
  
```

↓
Frontend가 access token 저장

3. 설계

- access token 기반 인증
- 이후 모든 trip API는 current_user를 기준으로 권한 검사
- 이메일/비밀번호 중심으로 서비스 설계

▼ 3.2. Globe Entry 아키텍처 (첫 진입화면)

1. 목적

- 앱 첫 화면에서 여행 감성 강화
- 사용자 여행을 지구본 위 핀으로 표시
- 핀 클릭 시 해당 여행 dashboard로 진입

2. 흐름

```
로그인 완료
↓
GET /trips/my
↓
FastAPI가 내가 owner/editor로 속한 trip 목록 조회
↓
Frontend가 각 trip의 대표 destination 좌표를 가져옴
↓
CesiumJS Globe에 marker/pin 표시
↓
사용자가 특정 trip 선택
↓
Trip Dashboard로 이동
```

3. 핵심 사항

- 3D globe는 수정 기능 없음
- DB에는 따로 저장 X: 단순 렌더링과 이동만 담당

▼ 3.3 Trip & Membership 아키텍처

1. 핵심 규칙

- 여행은 owner 1명이 소유

- editor는 공유받아 수정/편집 가능
- ownership 이전은 제외

2. 흐름

```

[여행 생성]
POST /trips
↓
FastAPI
├─ trips 생성
└─ trip_members에 owner 추가

[초대 생성]
POST /trips/{trip_id}/invites
↓
FastAPI
├─ owner/editor 권한 확인
├─ invite_code 생성
└─ invitations 저장

[초대 수락]
POST /invites/{code}/accept
↓
FastAPI
├─ code 유효성 검증
├─ trip_members 추가
└─ WebSocket: member_joined 전송

```

3. 권한 모델

- owner: 전체 관리 가능
- editor: shared resource 수정 가능
- personal expense(개인 소비)는 예외적으로 작성자만 보이며 수정 가능

▼ 3.4. Budget 아키텍처

1. budget 저장 흐름

```

사용자 예산 입력
↓
POST /trips/{trip_id}/budget
↓
FastAPI
├─ trip 권한 체크
└─ 총예산 저장

```

```
└─ 카테고리별 예산 저장
└─ budgets upsert
```

2. budget 요약 조회 흐름

```
GET /trips/{trip_id}/budget-summary
↓
FastAPI
└─ budget 조회
└─ shared + 내 personal expense 합산
└─ 카테고리별 소비 계산
└─ 남은 예산 계산
└─ overspending risk rule 계산
```

3. 과소비 분석 방식: rule-based로 깔끔하게 진행

- 전체 예산 사용률
- 여행 진행률 대비 소비 속도
- 카테고리별 과도한 사용
- 남은 일수 대비 예산 부족 가능성

→ dashboard로 warning만 보여주면됨

▼ 3.5. Expense 아키텍처

1. Expense 분류

- personal
- shared

2. 규칙

- personal → 작성자만 조회
- shared → trip member 전체 조회

3. 입력 흐름

```
사용자 소비 입력
↓
폼에서 personal/shared 선택
↓
POST /trips/{trip_id}/expenses
↓
FastAPI
```

```

└─ trip member 여부 확인
└─ expense_type 확인
└─ 환율 적용
└─ amount_krw 계산
└─ expenses 저장
└─ shared인 경우 WebSocket 이벤트 전송

```

4. 조회 흐름

```

GET /trips/{trip_id}/expenses
↓
FastAPI
└─ shared expense 조회
  └─ current_user의 personal expense 조회
↓
Frontend
└─ Shared Expenses 탭
  └─ My Personal Expenses 탭

```

5. 포인트: 같은 trip 안에서도 user별로 보이는 데이터가 다름!!!

→ 백엔드에서 필터링 강제해야함 (db로 강제하기)

▼ 3.6. OCR pipeline 아키텍처

1. 처리단계

- 이미지 업로드
- 파일 저장
- OCR 수행
- raw text 추출
- structured JSON 파싱
- 사용자 검수
- expense 확정 저장

2. 흐름

```

사용자가 영수증 업로드
↓
POST /trips/{trip_id}/receipts/upload
↓
FastAPI
└─ 파일 스토리지 저장

```

```

└─ receipts(status=processing) 생성
└─ AI Service 호출

AI Service
└─ OCR 수행
└─ raw text 생성
└─ parser로 amount/date/store/category 추출
└─ JSON 반환

FastAPI
└─ receipts 테이블 업데이트
└─ Frontend에 파싱 결과 반환

사용자
└─ 추출 결과 확인
└─ 수정 가능
└─ “expense로 저장” 클릭

POST /expenses/from-receipt
↓
expenses 생성

```

3. 이때 사용자 검수 단계가 반드시 필수적!!!

▼ 3.7. Exchange Rate 아키텍처

1. 핵심

- 백엔드에서 api로 호출 후 환율 결과 저장
- 리포트/예산 가격은 amount_krw 중심

2. 흐름

```

[매일 1회 Scheduler]
외부 환율 API 호출
↓
exchange_rates 저장

[expense 생성 시]
FastAPI
└─ 해당 일자의 최근 환율 조회
└─ 외화 → KRW 변환
└─ amount_krw 저장

```

▼ 3.8. Route/Places 아키텍처

1. 모듈 목적

- 여행 경로 기록
- 방문 장소 저장
- 순서 재배치
- 지도상 시각화
- Trip report용 장소 요약 데이터 제공

2. 흐름

```

사용자가 장소 추가
↓
POST /trips/{trip_id}/routes/{route_id}/places
↓
FastAPI
├─ 권한 체크
├─ place 저장
└─ visit_order 업데이트

사용자가 Route 페이지 진입
↓
GET /trips/{trip_id}/routes/{route_id}
↓
FastAPI
├─ route 조회
└─ route_places 조회
↓
Frontend
├─ Google Maps marker 표시
└─ polyline 렌더링

```

3. 저장 데이터

- place_name
- latitude / longitude
- address
- visit_order
- note
- planned_time or visited_time
- place_type
- city
- country

▼ 3.9. Collaboration / WebSocket 아키텍처

1. 실시간 처리 대상

- shared expense 생성/수정/삭제
- member join
- vote result update

2. WebSocket 구조

```
클라이언트가 trip 화면 진입
↓
/ws/trips/{trip_id} 연결
↓
FastAPI가 connection room 관리
↓
REST API에서 shared 데이터 변경 발생
↓
WebSocket event 브로드캐스트
↓
같은 trip room 참가자 UI 갱신
```

3. 중요한 원칙!!

- DB 저장은 REST
- 알림과 UI 갱신만 Websocket

▼ 3.10. Split/ 공동 소비 정산 아키텍처

자동 정산 아니고, 사용자 요청 시 정산 계산

```
shared expense 저장됨
↓
사용자가 "split 계산" 버튼 클릭
↓
POST /expenses/{expense_id}/split
↓
FastAPI
├─ split 대상 멤버 결정
├─ 1/N 계산
├─ expense_splits 저장 또는 계산 결과 반환
└─ 응답
```

▼ 3.11. Vote 아키텍처

```
투표 생성
↓
POST /trips/{trip_id}/votes
↓
vote + vote_options 저장
↓
WebSocket: vote_created

투표 참여
↓
POST /votes/{vote_id}/responses
↓
vote_responses 저장
↓
WebSocket: vote_updated
```

▼ 3.12. Trip Report 아키텍처

1. 목적

- 이번 여행을 정리
- 소비와 경로를 요약
- 여행의 특징을 텍스트로 설명
- 추억/인사이트 느낌 강화

2. 입력 데이터

- 여행 기간
- 목적지
- 총 소비액
- 카테고리별 소비 비율
- shared/personal 비율
- 방문 장소 수
- 경로 요약
- 대표 장소 유형

- 그룹 활동 정보

3. 생성 흐름

```

사용자가 "Generate Trip Report" 클릭
↓
POST /trips/{trip_id}/reports/generate
↓
FastAPI
├─ trip 데이터 aggregate
├─ expense summary 생성
├─ route summary 생성
├─ prompt 생성
└─ OpenAI API 호출
↓
trip_reports 저장
↓
Frontend 렌더링

```

4. 출력 예시

- 여행 요약
- 소비 패턴 분석
- 이번 여행의 분위기
- 인상적인 장소 유형
- 다음 여행 스타일 추천/ 추천 이유 (GPT prompt+summary 합치기)

아키텍처 별 역할분담

Aa 역할
<u>여행 생성 UI</u>
<u>내 여행 목록 화면</u>
<u>초대 코드 생성 UI</u>
<u>초대 코드 입력 UI</u>
<u>멤버 리스트 UI</u>
<u>trip 생성 API</u>
<u>trip_members 생성 및 관리</u>

Aa 역할
<u>invite code 생성 API</u>
<u>초대 수락 처리</u>
<u>trip DB 접근 권한 검증</u>
<u>Expense CRUD API</u>
<u>personal/shared 권한 필터링</u>
<u>환율 적용 및 amount_krw 계산</u>
<u>budget 저장 및 조회 API</u>
<u>budget vs expense 분석 로직</u>
<u>소비 입력 폼 (personal/shared 선택)</u>
<u>소비 리스트 UI (My/ shared 분리)</u>
<u>예산 입력 UI</u>
<u>예산/지출 그래프 시각화</u>
<u>카테고리 선택 UI</u>
<u>파일 업로드 처리(multipart)</u>
<u>파일 저장</u>
<u>AI service 호출</u>
<u>OCR 결과 저장</u>
<u>receipt → expense 변환 API</u>
<u>이미지 업로드 UI</u>
<u>OCR 결과 확인 화면</u>
<u>사용자 수정 UI</u>
<u>확정 저장 버튼 처리</u>
<u>route CRUD API</u>
<u>place CRUD API</u>
<u>visit_order 관리</u>
<u>경로 데이터 조회 API</u>
<u>Google Maps 연동</u>
<u>Marker 표시</u>
<u>polyline 경로 시각화</u>
<u>장소 리스트 UI</u>
<u>순서 변경 UI</u>

Aa 역할
<u>지도와 리스트 동기화</u>
<u>WebSocket 연결</u>
<u>이벤트 수신 처리</u>
<u>shared expense 자동 갱신</u>
<u>멤버 변경 UI 업데이트</u>
<u>Websocket 서버 구현</u>
<u>trip room 관리</u>
<u>이벤트 broadcast (expense, member, vote).</u>
<u>REST API와 연결</u>
<u>투표 생성 UI</u>
<u>투표 선택 UI</u>
<u>결과 시각화</u>
<u>실시간 결과 반영</u>
<u>vote 생성 API</u>
<u>response 저장 DB</u>
<u>vote 결과 집계</u>
<u>trip 데이터 aggregation</u>
<u>expense/category 요약</u>
<u>route 요약</u>
<u>prompt 생성</u>
<u>openai API 호출</u>
<u>report 저장</u>
<u>리포트 페이지 UI</u>
<u>Generate Report 버튼</u>
<u>텍스트 + 요약 시각화</u>
<u>로그인/회원가입 UI 구현</u>
<u>입력값 Validation</u>
<u>JWT 토큰 저장 (localStorage/cookie).</u>
<u>로그인 상태 관리 (global state).</u>
<u>로그인 후 페이지 이동 처리</u>
<u>회원가입 API 구현 (비밀번호 해시).</u>

Aa 역할
<u>로그인 API (비밀번호 검증 + JWT 발급).</u>
<u>현재 사용자 조회 API</u>
<u>인증 미들웨어 (JWT 검증).</u>

4. 전체 기능 흐름 시각화

4.1. 여행 관리 흐름

```

[Login]
  ↓
[Globe Entry]
  ↓
[Create Trip / Select Trip]
  ↓
[Invite Members]
  ↓
[Set Budget]
  ↓
[Record Expenses / Upload Receipts]
  ↓
[Manage Route in 2D Map]
  ↓
[Vote / Collaborate]
  ↓
[Generate Trip Report]
  ↓
[Generate User Recommendation Report]

```

4.2. expense 흐름

```

[Expense Input]
  ↓
personal or shared 선택
  ├── personal → 작성자 전용 조회
  └── shared → 그룹 공개
  ↓
환율 적용
  ↓
DB 저장

```

↓
(shared면) WebSocket 알림

4.3. OCR 흐름

[Upload Receipt]
↓
FastAPI 파일 저장
↓
AI Service OCR + Parsing
↓
추출 결과 반환
↓
사용자 검수
↓
Expense 저장

4.4 리포트 흐름

[Trip Report]
trip 데이터 집계
↓
prompt 생성
↓
LLM 호출
↓
trip_reports 저장

[Recommendation Report]
user 누적 trip 데이터 집계
↓
user_profiles 생성
↓
prompt 생성
↓
LLM 호출
↓
recommendation_reports 저장

5. 프론트페이지 구조

5.1. Login / Signup

- 인증 전용

5.2. Globe Entry

- 내 여행 핀 표시
- 여행 선택/생성

5.3. Trip Dashboard

- 예산 요약
- 최근 shared expense
- 최근 route preview
- recent activity
- report 진입

[Trip Dashboard]

↓

 Budget Summary
 Recent Expenses
 Route Preview
 Activity / Members
 Report Preview

[버튼]

→ Go to Expense
→ Go to Route
→ Go to Collaboration
→ Go to Report

5.4. Expense Page

- manual 입력
- OCR 업로드
- shared/personal 분리 조회

5.5. Route Page

- Google Maps 기반
- 장소 리스트 + 지도 동시 표시

5.6. Collaboration Page

- member list
- invite code
- vote
- activity stream

5.7. Trip Report Page

- 이번 여행 요약
- 소비/경로 분석
- 추천 여행지 & 추천 이유